BusNStay Multi-User Management System Integration Guide

Architecture Overview

User Roles & Hierarchy

```
Users (Base)
├── Passengers (Travelers)
├── Bus Operators
├── Delivery Agents
├── Restaurant Staff
│   └── Restaurant Admin
├── Hotel Staff
│   └── Hotel Manager
└── Platform Admins
```

Database Schema (Supabase PostgreSQL)

1. Core Users Table (Already in Supabase)

```sql
-- Base users table (if not exists)
CREATE TABLE IF NOT EXISTS users (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  email VARCHAR(255) UNIQUE NOT NULL,
  username VARCHAR(100) UNIQUE NOT NULL,
  phone_number VARCHAR(20),
  full_name VARCHAR(255) NOT NULL,
  password_hash VARCHAR(255) NOT NULL,
  profile_image_url TEXT,
  date_of_birth DATE,

  -- Account Status
  account_status ENUM('active', 'suspended', 'deleted', 'pending_verification')
DEFAULT 'pending_verification',
  is_email_verified BOOLEAN DEFAULT FALSE,
  is_phone_verified BOOLEAN DEFAULT FALSE,
  email_verified_at TIMESTAMPTZ,
  phone_verified_at TIMESTAMPTZ,
  last_login TIMESTAMPTZ,

  -- 2FA/Security
  two_factor_enabled BOOLEAN DEFAULT FALSE,
  two_factor_method ENUM('authenticator', 'sms', 'email') DEFAULT 'email',
  two_factor_secret VARCHAR(255),
```


Copyright (c) Zwesta Empire All rights reserved.

```
  backup_codes TEXT[],

  -- Account Settings
  notification_preferences JSONB DEFAULT '{"email": true, "sms": false, "push":
true}',
  language_preference VARCHAR(5) DEFAULT 'en',
  timezone VARCHAR(50) DEFAULT 'UTC',
  currency VARCHAR(3) DEFAULT 'ZMW',

  -- Preferences
  marketing_consent BOOLEAN DEFAULT FALSE,
  terms_accepted BOOLEAN DEFAULT FALSE,
  privacy_accepted BOOLEAN DEFAULT FALSE,

  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  deleted_at TIMESTAMPTZ,

  CONSTRAINT email_valid CHECK (email ~ '^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-
Z|a-z]{2,}$')
);

-- Indexes for performance
CREATE INDEX idx_users_email ON users(email);
CREATE INDEX idx_users_username ON users(username);
CREATE INDEX idx_users_phone ON users(phone_number);
CREATE INDEX idx_users_created_at ON users(created_at DESC);
CREATE INDEX idx_users_account_status ON users(account_status);

-- Auto-update trigger for updated_at
CREATE OR REPLACE FUNCTION update_users_timestamp()
RETURNS TRIGGER AS $$
BEGIN
  NEW.updated_at = now();
  RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER users_updated_at_trigger
BEFORE UPDATE ON users
FOR EACH ROW
EXECUTE FUNCTION update_users_timestamp();
```

2. User Roles & Permissions

```
-- User roles (many-to-many relationship)
CREATE TABLE IF NOT EXISTS user_roles (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  role ENUM(
```

 Copyright (c) Zwesta Empire All rights reserved.

```sql
    'passenger',
    'bus_operator',
    'delivery_agent',
    'restaurant_staff',
    'restaurant_admin',
    'hotel_staff',
    'hotel_manager',
    'platform_admin',
    'support_staff'
  ) NOT NULL,

  -- Role-specific metadata
  metadata JSONB,

  -- Role activation/deactivation
  is_active BOOLEAN DEFAULT TRUE,
  activated_at TIMESTAMPTZ DEFAULT now(),
  deactivated_at TIMESTAMPTZ,

  -- For operators/admins
  verification_status ENUM('pending', 'verified', 'rejected', 'suspended')
DEFAULT 'pending',
  verified_by UUID REFERENCES users(id),
  verified_at TIMESTAMPTZ,
  verification_documents JSONB,

  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT now(),

  UNIQUE(user_id, role)
);

CREATE INDEX idx_user_roles_user_id ON user_roles(user_id);
CREATE INDEX idx_user_roles_role ON user_roles(role);
CREATE INDEX idx_user_roles_is_active ON user_roles(is_active);

-- Permissions table
CREATE TABLE IF NOT EXISTS role_permissions (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  role VARCHAR(50) NOT NULL,
  permission VARCHAR(100) NOT NULL,
  description TEXT,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  UNIQUE(role, permission)
);

-- Sample permissions
INSERT INTO role_permissions (role, permission, description) VALUES
-- Passenger permissions
('passenger', 'view_routes', 'View available routes'),
('passenger', 'book_seat', 'Book a seat on a bus'),
('passenger', 'cancel_booking', 'Cancel their own bookings'),
('passenger', 'rate_trip', 'Rate completed trips'),
```

 Copyright (c) Zwesta Empire All rights reserved.

```
('passenger', 'order_food', 'Order from restaurants'),
('passenger', 'book_hotel', 'Book hotel accommodations'),
('passenger', 'make_payments', 'Make payments'),
('passenger', 'view_loyalty', 'View loyalty points'),
('passenger', 'use_wallet', 'Use wallet for payments'),

-- Bus Operator permissions
('bus_operator', 'manage_buses', 'Add/edit/delete buses'),
('bus_operator', 'manage_routes', 'Create and manage routes'),
('bus_operator', 'view_bookings', 'View passenger bookings'),
('bus_operator', 'view_revenue', 'View revenue reports'),
('bus_operator', 'manage_seats', 'Manage seat availability'),
('bus_operator', 'issue_refund', 'Issue refunds to passengers'),

-- Delivery Agent permissions
('delivery_agent', 'view_deliveries', 'View assigned deliveries'),
('delivery_agent', 'update_delivery_status', 'Update delivery status'),
('delivery_agent', 'track_location', 'Use location tracking'),
('delivery_agent', 'confirm_delivery', 'Confirm deliveries'),

-- Restaurant Admin permissions
('restaurant_admin', 'manage_menu', 'Manage menu items'),
('restaurant_admin', 'view_orders', 'View all restaurant orders'),
('restaurant_admin', 'manage_staff', 'Manage restaurant staff'),
('restaurant_admin', 'view_revenue', 'View restaurant revenue'),
('restaurant_admin', 'update_restaurant_info', 'Update restaurant info'),

-- Hotel Manager permissions
('hotel_manager', 'manage_rooms', 'Manage hotel rooms'),
('hotel_manager', 'manage_bookings', 'Manage room bookings'),
('hotel_manager', 'manage_staff', 'Manage hotel staff'),
('hotel_manager', 'view_revenue', 'View hotel revenue'),
('hotel_manager', 'update_hotel_info', 'Update hotel information'),

-- Platform Admin permissions
('platform_admin', 'manage_users', 'Manage all users'),
('platform_admin', 'view_analytics', 'View platform analytics'),
('platform_admin', 'manage_disputes', 'Handle disputes'),
('platform_admin', 'system_configuration', 'Configure system settings'),
('platform_admin', 'moderate_content', 'Moderate user content');
```

3. Session Management

```
-- User sessions table
CREATE TABLE IF NOT EXISTS user_sessions (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  token VARCHAR(500) UNIQUE NOT NULL,
  refresh_token VARCHAR(500) UNIQUE NOT NULL,
```

 Copyright (c) Zwesta Empire All rights reserved.

```
  -- Device info
  device_info JSONB,
  ip_address INET,
  user_agent TEXT,

  -- Security
  is_active BOOLEAN DEFAULT TRUE,
  is_revoked BOOLEAN DEFAULT FALSE,

  -- Timestamps
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  expires_at TIMESTAMPTZ NOT NULL DEFAULT (now() + INTERVAL '7 days'),
  last_activity TIMESTAMPTZ DEFAULT now(),
  revoked_at TIMESTAMPTZ,

  CONSTRAINT token_format CHECK (char_length(token) > 50)
);

CREATE INDEX idx_sessions_user_id ON user_sessions(user_id);
CREATE INDEX idx_sessions_token ON user_sessions(token);
CREATE INDEX idx_sessions_is_active ON user_sessions(is_active);
CREATE INDEX idx_sessions_expires_at ON user_sessions(expires_at);
```

4. User Profile Tables

```
-- Passenger profile
CREATE TABLE IF NOT EXISTS passenger_profiles (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID UNIQUE NOT NULL REFERENCES users(id) ON DELETE CASCADE,

  -- Preferences
  preferred_seats ENUM('window', 'aisle', 'any') DEFAULT 'any',
  accessibility_needs TEXT,
  emergency_contact_name VARCHAR(255),
  emergency_contact_phone VARCHAR(20),

  -- Travel preferences
  frequent_routes TEXT[],
  preferred_travel_times TIME[],

  -- Loyalty
  loyalty_points INTEGER DEFAULT 0,
  total_trips INTEGER DEFAULT 0,
  total_spent DECIMAL(10,2) DEFAULT 0,
  member_since TIMESTAMPTZ DEFAULT now(),
  loyalty_tier ENUM('bronze', 'silver', 'gold', 'platinum') DEFAULT 'bronze',

  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

 Copyright (c) Zwesta Empire All rights reserved.

```sql
-- Bus Operator profile
CREATE TABLE IF NOT EXISTS operator_profiles (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID UNIQUE NOT NULL REFERENCES users(id) ON DELETE CASCADE,

  -- Company info
  company_name VARCHAR(255) NOT NULL,
  company_registration_number VARCHAR(100),
  company_logo_url TEXT,
  company_address TEXT,

  -- Contact
  support_email VARCHAR(255),
  support_phone VARCHAR(20),

  -- Bank details
  bank_account_holder VARCHAR(255),
  bank_name VARCHAR(100),
  bank_account_number VARCHAR(50),
  bank_code VARCHAR(10),
  swift_code VARCHAR(11),

  -- Verification
  is_verified BOOLEAN DEFAULT FALSE,
  verification_documents JSONB,

  -- Statistics
  total_routes INTEGER DEFAULT 0,
  total_passengers INTEGER DEFAULT 0,
  average_rating DECIMAL(3,2),
  total_revenue DECIMAL(12,2) DEFAULT 0,

  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- Restaurant owner profile
CREATE TABLE IF NOT EXISTS restaurant_profiles (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID UNIQUE NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  restaurant_id UUID REFERENCES restaurants(id),

  -- Permissions
  can_manage_menu BOOLEAN DEFAULT TRUE,
  can_manage_orders BOOLEAN DEFAULT TRUE,
  can_manage_staff BOOLEAN DEFAULT FALSE,
  can_view_financials BOOLEAN DEFAULT FALSE,

  -- Contact
  position_title VARCHAR(100),

  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
```

 Copyright (c) Zwesta Empire All rights reserved.

```
  updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- Hotel manager profile
CREATE TABLE IF NOT EXISTS hotel_manager_profiles (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID UNIQUE NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  accommodation_id UUID REFERENCES accommodations(id),

  -- Permissions
  can_manage_rooms BOOLEAN DEFAULT TRUE,
  can_manage_bookings BOOLEAN DEFAULT TRUE,
  can_manage_staff BOOLEAN DEFAULT FALSE,
  can_view_financials BOOLEAN DEFAULT FALSE,

  -- Contact
  position_title VARCHAR(100),

  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

5. Account Verification & KYC

```
-- Email verification
CREATE TABLE IF NOT EXISTS email_verifications (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  email VARCHAR(255) NOT NULL,
  verification_code VARCHAR(6) NOT NULL,

  is_verified BOOLEAN DEFAULT FALSE,
  verified_at TIMESTAMPTZ,

  attempts INTEGER DEFAULT 0,
  max_attempts INTEGER DEFAULT 5,

  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  expires_at TIMESTAMPTZ NOT NULL DEFAULT (now() + INTERVAL '24 hours'),

  CONSTRAINT valid_code CHECK (char_length(verification_code) = 6)
);

-- Phone verification
CREATE TABLE IF NOT EXISTS phone_verifications (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  phone_number VARCHAR(20) NOT NULL,
  verification_code VARCHAR(6) NOT NULL,
```

 Copyright (c) Zwesta Empire All rights reserved.

```
  is_verified BOOLEAN DEFAULT FALSE,
  verified_at TIMESTAMPTZ,

  attempts INTEGER DEFAULT 0,
  max_attempts INTEGER DEFAULT 5,

  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  expires_at TIMESTAMPTZ NOT NULL DEFAULT (now() + INTERVAL '10 minutes'),

  CONSTRAINT valid_code CHECK (char_length(verification_code) = 6)
);

-- KYC (Know Your Customer) documents
CREATE TABLE IF NOT EXISTS kyc_documents (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,

  document_type ENUM('national_id', 'passport', 'driver_license',
'business_license') NOT NULL,
  document_number VARCHAR(100) NOT NULL,
  document_url TEXT NOT NULL,

  -- Verification
  verification_status ENUM('pending', 'approved', 'rejected', 'expired')
DEFAULT 'pending',
  verified_by UUID REFERENCES users(id),
  verified_at TIMESTAMPTZ,
  verification_notes TEXT,

  -- Expiry
  issued_date DATE,
  expiry_date DATE,

  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT now(),

  UNIQUE(user_id, document_type)
);
```

6. User Activity & Audit Logging

```
-- Audit log for all user changes
CREATE TABLE IF NOT EXISTS audit_logs (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID REFERENCES users(id) ON DELETE SET NULL,

  action VARCHAR(100) NOT NULL,
  entity_type VARCHAR(50) NOT NULL,
  entity_id UUID,
```

 Copyright (c) Zwesta Empire All rights reserved.

```
  old_values JSONB,
  new_values JSONB,
  changes JSONB,

  ip_address INET,
  user_agent TEXT,

  created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE INDEX idx_audit_user_id ON audit_logs(user_id);
CREATE INDEX idx_audit_entity ON audit_logs(entity_type, entity_id);
CREATE INDEX idx_audit_created_at ON audit_logs(created_at DESC);
```

Frontend Implementation

1. TypeScript Types

```
// src/types/auth.types.ts
export type UserRole =
  | 'passenger'
  | 'bus_operator'
  | 'delivery_agent'
  | 'restaurant_staff'
  | 'restaurant_admin'
  | 'hotel_staff'
  | 'hotel_manager'
  | 'platform_admin'
  | 'support_staff';

export type AccountStatus = 'active' | 'suspended' | 'deleted' |
'pending_verification';

export type VerificationStatus = 'pending' | 'verified' | 'rejected' |
'suspended';

export interface User {
  id: string;
  email: string;
  username: string;
  full_name: string;
  phone_number?: string;
  profile_image_url?: string;
  date_of_birth?: string;
  account_status: AccountStatus;
  is_email_verified: boolean;
  is_phone_verified: boolean;
  two_factor_enabled: boolean;
  created_at: string;
```

 Copyright (c) Zwesta Empire All rights reserved.

```
  updated_at: string;
}

export interface UserRole {
  id: string;
  user_id: string;
  role: UserRole;
  is_active: boolean;
  verification_status: VerificationStatus;
  metadata?: Record<string, any>;
  created_at: string;
}

export interface AuthSession {
  user: User;
  roles: UserRole[];
  permissions: string[];
  token: string;
  refresh_token: string;
  expires_at: string;
}

export interface RegisterPayload {
  email: string;
  password: string;
  confirm_password: string;
  full_name: string;
  phone_number?: string;
  date_of_birth?: string;
  terms_accepted: boolean;
  privacy_accepted: boolean;
  referral_code?: string;
}

export interface LoginPayload {
  email: string;
  password: string;
}

export interface CreateOperatorPayload extends RegisterPayload {
  company_name: string;
  company_registration_number: string;
  support_email: string;
  support_phone: string;
  bank_account_holder: string;
  bank_name: string;
  bank_account_number: string;
  bank_code: string;
}
```

2. Authentication Service

 Copyright (c) Zwesta Empire All rights reserved.

```
// src/services/AuthService.ts
import { createClient } from '@supabase/supabase-js';

const supabase = createClient(
  import.meta.env.VITE_SUPABASE_URL,
  import.meta.env.VITE_SUPABASE_PUBLISHABLE_KEY
);

export class AuthService {
  /**
   * Register new user
   */
  static async register(payload: RegisterPayload) {
    try {
      // 1. Create auth user in Supabase Auth
      const { data: authData, error: authError } = await supabase.auth.signUp({
        email: payload.email,
        password: payload.password,
      });

      if (authError) throw authError;

      const userId = authData.user?.id;

      // 2. Create user profile in database
      const { data: user, error: userError } = await supabase
        .from('users')
        .insert({
          id: userId,
          email: payload.email,
          username: payload.email.split('@')[0],
          full_name: payload.full_name,
          phone_number: payload.phone_number,
          date_of_birth: payload.date_of_birth,
          terms_accepted: payload.terms_accepted,
          privacy_accepted: payload.privacy_accepted,
        })
        .select()
        .single();

      if (userError) throw userError;

      // 3. Assign default role (passenger)
      await supabase.from('user_roles').insert({
        user_id: userId,
        role: 'passenger',
        is_active: true,
      });

      // 4. Create passenger profile
      await supabase.from('passenger_profiles').insert({
```

 Copyright (c) Zwesta Empire All rights reserved.

```
        user_id: userId,
        member_since: new Date().toISOString(),
      });

      // 5. Send verification email
      await this.sendVerificationEmail(userId, payload.email);

      return { user, success: true };
    } catch (error) {
      console.error('Registration error:', error);
      throw error;
    }
  }

  /**
   * Send verification email
   */
  static async sendVerificationEmail(userId: string, email: string) {
    try {
      // Generate 6-digit code
      const code = Math.floor(100000 + Math.random() * 900000).toString();

      // Store in database
      await supabase.from('email_verifications').insert({
        user_id: userId,
        email,
        verification_code: code,
        expires_at: new Date(Date.now() + 24 * 60 * 60 * 1000).toISOString(),
      });

      // Send email (integrate with Resend, SendGrid, etc.)
      // Example: await sendEmail(email, 'Verify your BusNStay account', `Your
code: ${code}`);

      console.log(`Verification code sent to ${email}: ${code}`);
    } catch (error) {
      console.error('Error sending verification email:', error);
      throw error;
    }
  }

  /**
   * Verify email address
   */
  static async verifyEmail(userId: string, code: string) {
    try {
      // Check if code is valid
      const { data: verification, error } = await supabase
        .from('email_verifications')
        .select()
        .eq('user_id', userId)
        .eq('verification_code', code)
        .eq('is_verified', false)
```

 Copyright (c) Zwesta Empire All rights reserved.

```
        .gt('expires_at', new Date().toISOString())
        .single();

      if (error || !verification) {
        throw new Error('Invalid or expired verification code');
      }

      // Check attempts
      if (verification.attempts >= verification.max_attempts) {
        throw new Error('Too many attempts. Please request a new code.');
      }

      // Mark as verified
      await supabase
        .from('email_verifications')
        .update({
          is_verified: true,
          verified_at: new Date().toISOString(),
        })
        .eq('id', verification.id);

      // Update user
      await supabase
        .from('users')
        .update({
          is_email_verified: true,
          email_verified_at: new Date().toISOString(),
        })
        .eq('id', userId);

      return { success: true };
    } catch (error) {
      console.error('Email verification error:', error);
      throw error;
    }
  }

  /**
   * Login user
   */
  static async login(email: string, password: string) {
    try {
      // Authenticate with Supabase
      const { data, error } = await supabase.auth.signInWithPassword({
        email,
        password,
      });

      if (error) throw error;

      // Fetch user profile
      const { data: user } = await supabase
        .from('users')
```

 Copyright (c) Zwesta Empire All rights reserved.

```
        .select()
        .eq('id', data.user.id)
        .single();

      // Fetch user roles
      const { data: roles } = await supabase
        .from('user_roles')
        .select()
        .eq('user_id', data.user.id)
        .eq('is_active', true);

      // Fetch permissions for all roles
      const roleNames = roles?.map((r) => r.role) || ['passenger'];
      const { data: permissions } = await supabase
        .from('role_permissions')
        .select('permission')
        .in('role', roleNames);

      const permissionList = permissions?.map((p) => p.permission) || [];

      // Create session
      const session: AuthSession = {
        user,
        roles,
        permissions: permissionList,
        token: data.session?.access_token,
        refresh_token: data.session?.refresh_token,
        expires_at: data.session?.expires_at,
      };

      // Store in localStorage
      localStorage.setItem('auth_session', JSON.stringify(session));

      return session;
    } catch (error) {
      console.error('Login error:', error);
      throw error;
    }
  }

  /**
   * Logout user
   */
  static async logout() {
    try {
      await supabase.auth.signOut();
      localStorage.removeItem('auth_session');
      return { success: true };
    } catch (error) {
      console.error('Logout error:', error);
      throw error;
    }
  }
```

 Copyright (c) Zwesta Empire All rights reserved.

```
  /**
   * Add role to user (Admin action)
   */
  static async addRoleToUser(userId: string, role: UserRole) {
    try {
      const { data, error } = await supabase
        .from('user_roles')
        .insert({
          user_id: userId,
          role,
          is_active: true,
        })
        .select()
        .single();

      if (error) throw error;

      return data;
    } catch (error) {
      console.error('Error adding role:', error);
      throw error;
    }
  }

  /**
   * Check if user has permission
   */
  static hasPermission(session: AuthSession, permission: string): boolean {
    return session.permissions.includes(permission);
  }

  /**
   * Get all user permissions
   */
  static getPermissions(session: AuthSession): string[] {
    return session.permissions;
  }
}
```

3. Auth Context

```
// src/contexts/AuthContext.tsx
import React, { createContext, useState, useEffect, ReactNode } from 'react';
import { AuthService } from '../services/AuthService';

export const AuthContext = createContext<{
  session: AuthSession | null;
  isLoading: boolean;
  login: (email: string, password: string) => Promise<void>;
```

 Copyright (c) Zwesta Empire All rights reserved.

```
  register: (payload: RegisterPayload) => Promise<void>;
  logout: () => Promise<void>;
  hasPermission: (permission: string) => boolean;
  hasRole: (role: UserRole) => boolean;
}>({
  session: null,
  isLoading: false,
  login: async () => {},
  register: async () => {},
  logout: async () => {},
  hasPermission: () => false,
  hasRole: () => false,
});

export const AuthProvider: React.FC<{ children: ReactNode }> = ({
  children,
}) => {
  const [session, setSession] = useState<AuthSession | null>(null);
  const [isLoading, setIsLoading] = useState(true);

  // Load session on mount
  useEffect(() => {
    const stored = localStorage.getItem('auth_session');
    if (stored) {
      setSession(JSON.parse(stored));
    }
    setIsLoading(false);
  }, []);

  const login = async (email: string, password: string) => {
    setIsLoading(true);
    try {
      const newSession = await AuthService.login(email, password);
      setSession(newSession);
    } finally {
      setIsLoading(false);
    }
  };

  const register = async (payload: RegisterPayload) => {
    setIsLoading(true);
    try {
      await AuthService.register(payload);
      // Show verification UI
    } finally {
      setIsLoading(false);
    }
  };

  const logout = async () => {
    setIsLoading(true);
    try {
      await AuthService.logout();
```

 Copyright (c) Zwesta Empire All rights reserved.

```
      setSession(null);
    } finally {
      setIsLoading(false);
    }
  };

  const hasPermission = (permission: string): boolean => {
    if (!session) return false;
    return AuthService.hasPermission(session, permission);
  };

  const hasRole = (role: UserRole): boolean => {
    if (!session) return false;
    return session.roles.some((r) => r.role === role && r.is_active);
  };

  return (
    <AuthContext.Provider
      value={{
        session,
        isLoading,
        login,
        register,
        logout,
        hasPermission,
        hasRole,
      }}
    >
      {children}
    </AuthContext.Provider>
  );
};

export const useAuth = () => {
  const context = React.useContext(AuthContext);
  if (!context) {
    throw new Error('useAuth must be used within AuthProvider');
  }
  return context;
};
```

4. Protected Route Component

```
// src/components/ProtectedRoute.tsx
import React from 'react';
import { Navigate } from 'react-router-dom';
import { useAuth } from '../contexts/AuthContext';

interface ProtectedRouteProps {
  children: React.ReactNode;
```

 Copyright (c) Zwesta Empire All rights reserved.

```
  requiredRole?: UserRole;
  requiredPermission?: string;
}

export const ProtectedRoute: React.FC<ProtectedRouteProps> = ({
  children,
  requiredRole,
  requiredPermission,
}) => {
  const { session, isLoading } = useAuth();

  if (isLoading) {
    return <div className="flex items-center justify-center min-h-
screen">Loading...</div>;
  }

  if (!session) {
    return <Navigate to="/login" replace />;
  }

  if (requiredRole && !session.roles.some((r) => r.role === requiredRole &&
r.is_active)) {
    return <Navigate to="/unauthorized" replace />;
  }

  if (requiredPermission && !session.permissions.includes(requiredPermission))
{
    return <Navigate to="/unauthorized" replace />;
  }

  return <>{children}</>;
};
```

Registration Flows

1. Passenger Registration

```
// src/pages/auth/PassengerRegistration.tsx
export const PassengerRegistration = () => {
  const { register } = useAuth();
  const [formData, setFormData] = useState<RegisterPayload>({
    email: '',
    password: '',
    confirm_password: '',
    full_name: '',
    phone_number: '',
    date_of_birth: '',
    terms_accepted: false,
    privacy_accepted: false,
```

 Copyright (c) Zwesta Empire All rights reserved.

```
  });

  const handleSubmit = async (e: React.FormEvent) => {
    e.preventDefault();

    if (formData.password !== formData.confirm_password) {
      alert('Passwords do not match');
      return;
    }

    try {
      await register(formData);
      // Navigate to email verification
    } catch (error) {
      alert('Registration failed: ' + error.message);
    }
  };

  return (
    <form onSubmit={handleSubmit} className="max-w-md mx-auto space-y-4">
      <input
        type="email"
        placeholder="Email"
        value={formData.email}
        onChange={(e) => setFormData({ ...formData, email: e.target.value })}
        required
      />
      <input
        type="text"
        placeholder="Full Name"
        value={formData.full_name}
        onChange={(e) => setFormData({ ...formData, full_name: e.target.value
})}
        required
      />
      <input
        type="tel"
        placeholder="Phone Number"
        value={formData.phone_number}
        onChange={(e) => setFormData({ ...formData, phone_number:
e.target.value })}
      />
      <input
        type="password"
        placeholder="Password"
        value={formData.password}
        onChange={(e) => setFormData({ ...formData, password: e.target.value
})}
        required
      />
      <input
        type="password"
        placeholder="Confirm Password"
```

 Copyright (c) Zwesta Empire All rights reserved.

```
        value={formData.confirm_password}
        onChange={(e) => setFormData({ ...formData, confirm_password:
e.target.value })}
        required
      />
      <label className="flex items-center gap-2">
        <input
          type="checkbox"
          checked={formData.terms_accepted}
          onChange={(e) => setFormData({ ...formData, terms_accepted:
e.target.checked })}
          required
        />
        I agree to Terms of Service
      </label>
      <button type="submit" className="w-full bg-blue-600 text-white py-2
rounded">
        Register
      </button>
    </form>
  );
};
```

2. Bus Operator Registration

```
// src/pages/auth/BusOperatorRegistration.tsx
export const BusOperatorRegistration = () => {
  const { register } = useAuth();
  const [formData, setFormData] = useState<CreateOperatorPayload>({
    email: '',
    password: '',
    confirm_password: '',
    full_name: '',
    phone_number: '',
    company_name: '',
    company_registration_number: '',
    support_email: '',
    support_phone: '',
    bank_account_holder: '',
    bank_name: '',
    bank_account_number: '',
    bank_code: '',
    terms_accepted: false,
    privacy_accepted: false,
  });

  const handleSubmit = async (e: React.FormEvent) => {
    e.preventDefault();
    try {
      await register(formData);
```

 Copyright (c) Zwesta Empire All rights reserved.

```
      // Navigate to KYC verification
      alert('Application submitted. Awaiting verification.');
    } catch (error) {
      alert('Registration failed: ' + error.message);
    }
  };

  return (
    <form onSubmit={handleSubmit} className="max-w-2xl mx-auto space-y-4">
      {/* Personal Info */}
      <h2 className="text-xl font-bold">Personal Information</h2>
      <input
        type="email"
        placeholder="Email"
        value={formData.email}
        onChange={(e) => setFormData({ ...formData, email: e.target.value })}
        required
      />
      <input
        type="text"
        placeholder="Full Name"
        value={formData.full_name}
        onChange={(e) => setFormData({ ...formData, full_name: e.target.value
})}
        required
      />

      {/* Company Info */}
      <h2 className="text-xl font-bold mt-6">Company Information</h2>
      <input
        type="text"
        placeholder="Company Name"
        value={formData.company_name}
        onChange={(e) => setFormData({ ...formData, company_name:
e.target.value })}
        required
      />
      <input
        type="text"
        placeholder="Company Registration Number"
        value={formData.company_registration_number}
        onChange={(e) =>
          setFormData({ ...formData, company_registration_number:
e.target.value })
        }
        required
      />

      {/* Bank Details */}
      <h2 className="text-xl font-bold mt-6">Bank Details</h2>
      <input
        type="text"
        placeholder="Bank Account Holder"
```

 Copyright (c) Zwesta Empire All rights reserved.

```
          value={formData.bank_account_holder}
          onChange={(e) =>
            setFormData({ ...formData, bank_account_holder: e.target.value })
          }
          required
        />
        <input
          type="text"
          placeholder="Bank Account Number"
          value={formData.bank_account_number}
          onChange={(e) =>
            setFormData({ ...formData, bank_account_number: e.target.value })
          }
          required
        />

        <button type="submit" className="w-full bg-blue-600 text-white py-2
rounded">
          Submit Application
        </button>
      </form>
    );
  };
```

User Account Management

1. Profile Management Page

```
// src/pages/account/ProfileManagement.tsx
export const ProfileManagement = () => {
  const { session } = useAuth();
  const [user, setUser] = useState(session?.user);
  const [isEditing, setIsEditing] = useState(false);

  const updateProfile = async (updates: Partial<User>) => {
    try {
      const { data, error } = await supabase
        .from('users')
        .update(updates)
        .eq('id', session!.user.id)
        .select()
        .single();

      if (error) throw error;
      setUser(data);
      alert('Profile updated successfully');
    } catch (error) {
      alert('Update failed: ' + error.message);
    }
```

 Copyright (c) Zwesta Empire All rights reserved.

```
  };

  return (
    <div className="max-w-2xl mx-auto p-6">
      <h1 className="text-2xl font-bold mb-6">Profile Management</h1>

      <div className="bg-white rounded-lg shadow p-6 space-y-4">
        {/* Profile Picture */}
        <div>
          <label>Profile Picture</label>
          <img
            src={user?.profile_image_url || '/default-avatar.png'}
            alt="Profile"
            className="w-24 h-24 rounded-full"
          />
          {isEditing && (
            <input type="file" accept="image/*" onChange={(e) => {}} />
          )}
        </div>

        {/* Personal Info */}
        <div>
          <label>Full Name</label>
          <input
            type="text"
            value={user?.full_name || ''}
            onChange={(e) => setUser({ ...user!, full_name: e.target.value })}
            disabled={!isEditing}
          />
        </div>

        <div>
          <label>Email</label>
          <input
            type="email"
            value={user?.email || ''}
            disabled
            className="bg-gray-100"
          />
        </div>

        <div>
          <label>Phone</label>
          <input
            type="tel"
            value={user?.phone_number || ''}
            onChange={(e) => setUser({ ...user!, phone_number: e.target.value
})}
            disabled={!isEditing}
          />
        </div>

        {/* Verification Status */}
```

 Copyright (c) Zwesta Empire All rights reserved.

```
        <div className="border-t pt-4">
          <h3 className="font-bold mb-2">Verification Status</h3>
          <p>Email: {user?.is_email_verified ? '✅ Verified' : '❌ Not
Verified'}</p>
          <p>Phone: {user?.is_phone_verified ? '✅ Verified' : '❌ Not
Verified'}</p>
        </div>

        {/* 2FA */}
        <div className="border-t pt-4">
          <label className="flex items-center gap-2">
            <input
              type="checkbox"
              checked={user?.two_factor_enabled || false}
              onChange={(e) =>
                setUser({ ...user!, two_factor_enabled: e.target.checked })
              }
            />
            Enable 2-Factor Authentication
          </label>
        </div>

        {/* Buttons */}
        <div className="flex gap-2 pt-4">
          {!isEditing ? (
            <button
              onClick={() => setIsEditing(true)}
              className="bg-blue-600 text-white px-4 py-2 rounded"
            >
              Edit Profile
            </button>
          ) : (
            <>
              <button
                onClick={() => updateProfile(user!)}
                className="bg-green-600 text-white px-4 py-2 rounded"
              >
                Save Changes
              </button>
              <button
                onClick={() => {
                  setUser(session?.user);
                  setIsEditing(false);
                }}
                className="bg-gray-600 text-white px-4 py-2 rounded"
              >
                Cancel
              </button>
            </>
          )}
        </div>
      </div>
    </div>
```

 Copyright (c) Zwesta Empire All rights reserved.

```
  );
};
```

Supabase RLS Policies

```sql
-- Enable RLS on all tables
ALTER TABLE users ENABLE ROW LEVEL SECURITY;
ALTER TABLE user_roles ENABLE ROW LEVEL SECURITY;
ALTER TABLE user_sessions ENABLE ROW LEVEL SECURITY;
ALTER TABLE email_verifications ENABLE ROW LEVEL SECURITY;

-- Users can view their own profile
CREATE POLICY "users_view_own_profile" ON users
  FOR SELECT USING (auth.uid() = id);

-- Users can update their own profile
CREATE POLICY "users_update_own_profile" ON users
  FOR UPDATE USING (auth.uid() = id);

-- Users can view their own roles
CREATE POLICY "users_view_own_roles" ON user_roles
  FOR SELECT USING (auth.uid() = user_id);

-- Platform admins can view all users
CREATE POLICY "admins_view_all_users" ON users
  FOR SELECT USING (
    EXISTS (
      SELECT 1 FROM user_roles
      WHERE user_roles.user_id = auth.uid()
        AND user_roles.role = 'platform_admin'
        AND user_roles.is_active = TRUE
    )
  );

-- Email verifications - users can view their own
CREATE POLICY "users_view_own_verification" ON email_verifications
  FOR SELECT USING (auth.uid() = user_id);
```

Deployment Checklist

- ☐ Create all database tables in Supabase
- ☐ Enable PostGIS extension (if needed)
- ☐ Set up RLS policies
- ☐ Configure Supabase Auth (Email confirmation, Password reset)
- ☐ Set up email service (Resend/SendGrid)
- ☐ Implement frontend auth flows

Copyright (c) Zwesta Empire All rights reserved.

- ☐ Test registration with all user types
- ☐ Test login flow
- ☐ Test 2FA setup
- ☐ Test role-based access control
- ☐ Set up audit logging
- ☐ Deploy to Vercel
- ☐ Document API endpoints
- ☐ Create admin panel for user management

Next Steps

1. **Update Database**: Execute all SQL schemas in this guide
2. **Implement Services**: Create AuthService and related utilities
3. **Build UI Components**: Create registration, login, and profile pages
4. **Test Thoroughly**: Test all registration flows and permission checks
5. **Deploy**: Push to Vercel and test in production
6. **Monitor**: Set up logging and monitoring for auth events

Copyright (c) Zwesta Empire All rights reserved.